

SIMATS ENGINEERING
ASSESSMENT TOOL 3 — ANNEXURE A

Technical Presentation Solutions

Intelligent Agent Architectures using PySpark, RDDs
and Distributed AI Systems

Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO5 — Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures for distributed AI applications (BTL 5)
Assessment	Technical Presentation 5 Questions Total Marks: 50

This document presents complete, rubric-aligned solutions for all five technical presentation topics listed in Annexure A. Each solution covers problem identification, agent architecture and design, integration of PySpark / RDD-based distributed data processing, tools used, an architecture diagram, and a conclusion — addressing the evaluation criteria: Technical Content Accuracy, Problem Identification, Use of Tools, Communication, and Professionalism.

Contents

- 1. Reactive Agent-Based Traffic Control System**
- 2. Goal-Based Agent for Healthcare Diagnosis**
- 3. Utility-Based Agent for E-Commerce Recommendations**
- 4. Learning Agent for Personalized Education System**
- 5. Multi-Agent System for Disaster Management**
- 6. Rubric Alignment Summary**

1. Reactive Agent-Based Traffic Control System

1.1 Problem Identification

Urban traffic congestion arises because fixed-timer signals cannot adapt to real-time changes in vehicle density, pedestrian flow, or emergency situations. The engineering problem is to design an agent that senses live traffic conditions at an intersection and reacts instantly — without complex internal reasoning about future states — to adjust signal timing and minimize congestion. This fits a **reactive (stimulus–response) agent** because decisions must be made in milliseconds directly from current percepts.

1.2 Agent Architecture

A reactive agent operates purely on a condition–action (if–then) rule set, with no memory of history and no internal model of the world. The agent's percept-to-action cycle is:

- **Percepts:** vehicle count per lane, average speed, pedestrian presence, and emergency-vehicle signals from IoT cameras and inductive loop sensors.
- **Rule base:** e.g. *IF* queue_length(lane) > threshold *THEN* extend_green(lane); *IF* emergency_vehicle_detected *THEN* preempt_signal.
- **Actuators:** traffic-light phase controller and variable message signs for rerouting.
- **No planning/learning:** the agent's behaviour is a direct mapping from percept to action, making it extremely fast and robust for real-time control.

1.3 Role of PySpark, RDDs and Streaming Data

A single intersection generates a continuous stream of sensor readings, and a city-wide deployment covers hundreds of junctions simultaneously — this is where distributed processing becomes essential. Incoming sensor events are ingested through a streaming layer (e.g. Spark Structured Streaming / Kafka) and converted into **micro-batch RDDs**. PySpark's map() and reduceByKey() transformations aggregate vehicle counts per lane, per junction, across the whole city in parallel, so the condition-action rules evaluate on fresh, city-scale data within sub-second latency. Partitioning RDDs by junction ID allows each partition to be processed independently on a different executor node, giving linear scalability as more intersections are added.

1.4 Tools Used

- Apache Spark (PySpark) Structured Streaming for real-time RDD/DataFrame processing
- IoT sensors / traffic cameras for percept acquisition
- Kafka (or similar) as the streaming ingestion layer
- A rule engine (Drools-style or simple Python condition set) for the reactive logic

Reactive Agent Architecture — Smart Traffic Control

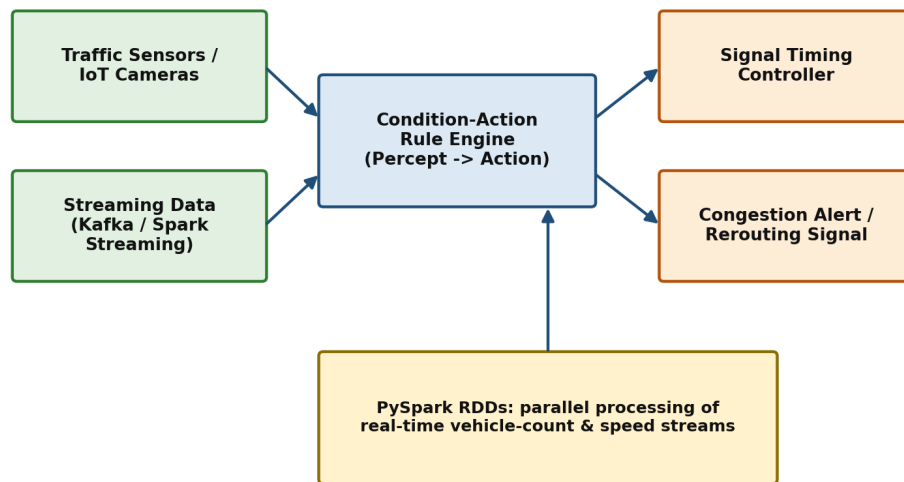


Fig. 1 — Reactive agent perception–action loop for smart traffic control.

1.5 Conclusion

The reactive agent design is well suited to traffic control because it guarantees fast, low-latency responses at the level of a single intersection, while PySpark's distributed RDD processing lets the same architecture scale to an entire city's sensor network without redesigning the underlying decision logic.

2. Goal-Based Agent for Healthcare Diagnosis

2.1 Problem Identification

Diagnosing a patient is not a single reflex action — it requires evaluating multiple possible conditions against symptoms, history, and lab results, then selecting the path that leads to the explicit **goal** of a correct, confirmed diagnosis. A purely reactive agent cannot do this because it has no way to represent "what state I want to reach." The problem therefore calls for a **goal-based agent** that models future states and reasons about which sequence of tests or actions achieves the diagnostic goal.

2.2 Agent Architecture

- **State representation:** current patient state built from EHR data, vitals, symptoms, and lab results.
- **Goal:** reach a state where the diagnosis is confirmed with sufficient confidence (e.g. probability above a clinical threshold).
- **Search / planning:** the agent explores candidate diagnostic pathways (which tests to order next, which conditions to rule out) using search strategies such as best-first search over a decision tree of hypotheses.
- **Action selection:** chooses the next diagnostic action (order test, refer specialist, recommend treatment) that reduces the distance to the goal state.
- **Model of the world:** unlike a reactive agent, it maintains an internal model connecting symptoms, tests and probable conditions.

2.3 Role of PySpark, RDDs and Distributed Processing

Hospitals generate millions of patient records across departments. Before the agent can reason over an individual case, the system must extract relevant features (lab trends, comorbidity patterns, historical outcomes for similar patients) from this large corpus. PySpark RDDs are used to **parallelize feature extraction and cohort comparison**: patient records are loaded as RDDs, filtered and transformed (map, filter, join) across distributed nodes, and aggregated (reduceByKey) to compute statistics such as symptom-to-diagnosis correlation at population scale. These aggregated features are then supplied to the goal-based reasoning module so its decisions are grounded in large-scale evidence rather than a single patient's data alone.

2.4 Tools Used

- PySpark RDDs / DataFrames for distributed EHR data processing
- A planning module (search over diagnostic hypothesis graph)
- Clinical knowledge base / ontology (e.g. SNOMED-style condition mapping)
- Data pipeline connecting hospital EHR systems to the Spark cluster

Goal-Based Agent Architecture — Healthcare Diagnosis

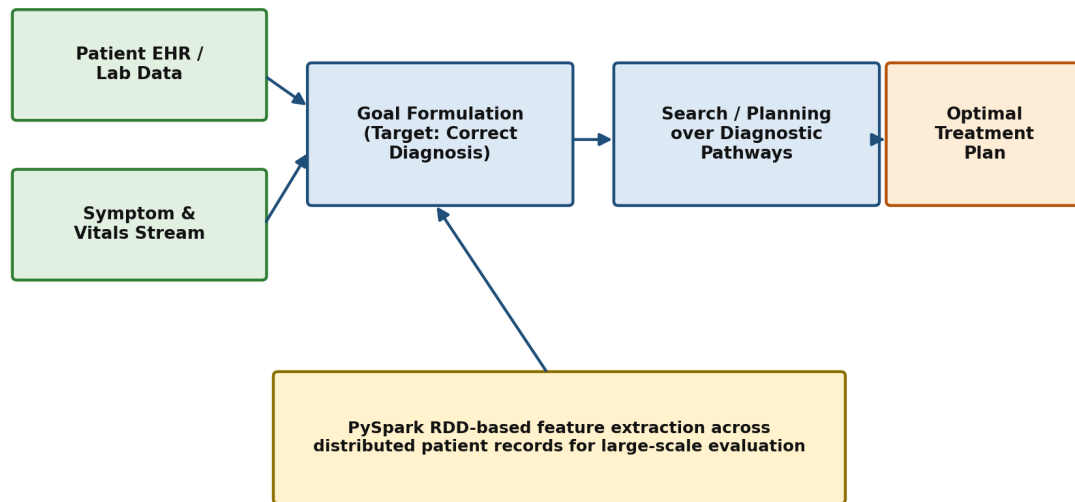


Fig. 2 — Goal-based agent reasoning pipeline for diagnostic decision-making.

2.5 Conclusion

By combining explicit goal representation with distributed data processing, the agent can reason about the best path to a correct diagnosis while grounding every decision in evidence drawn from population-scale patient data — something a stimulus-response agent could never achieve.

3. Utility-Based Agent for E-Commerce Recommendations

3.1 Problem Identification

In e-commerce, multiple products can satisfy the same customer goal (e.g. "buy a laptop"), but they differ in price, rating, relevance and delivery time. A goal-based agent that only checks "goal satisfied / not satisfied" cannot distinguish between these options. The problem calls for a **utility-based agent** that assigns a numeric desirability score to each candidate option and selects the one that maximizes expected customer satisfaction.

3.2 Agent Architecture

- **Candidate generation:** retrieve products matching the user's query/history from the catalog.
- **Utility function:** $U(\text{item}) = w_1 \cdot \text{Relevance} + w_2 \cdot (1/\text{Price}) + w_3 \cdot \text{Rating} + w_4 \cdot \text{Availability}$, with weights tuned per user segment.
- **Ranking:** candidates are scored and sorted by utility; the highest-utility items are presented first.
- **Feedback loop:** click-through and purchase data refine the weights over time, improving personalization.

3.3 Role of PySpark, RDDs and Large-Scale Data

A large e-commerce platform must score millions of products against millions of users in near real time. Product and user-interaction data are represented as PySpark RDDs, partitioned by product category or user segment. The utility computation is expressed as a `map()` transformation applied in parallel across partitions, and `reduceByKey()/sortBy()` operations aggregate and rank the top-N items per user across the cluster. This distributed approach is what makes personalized, utility-based recommendation feasible at the scale of a real online store, where a single-machine computation would be far too slow.

3.4 Tools Used

- PySpark RDDs / MLlib for distributed utility scoring and ranking
- Collaborative-filtering or content-based feature store
- A/B testing framework to validate utility-function weights
- Real-time recommendation API serving the ranked list

Utility-Based Agent Architecture — E-Commerce Recommendations

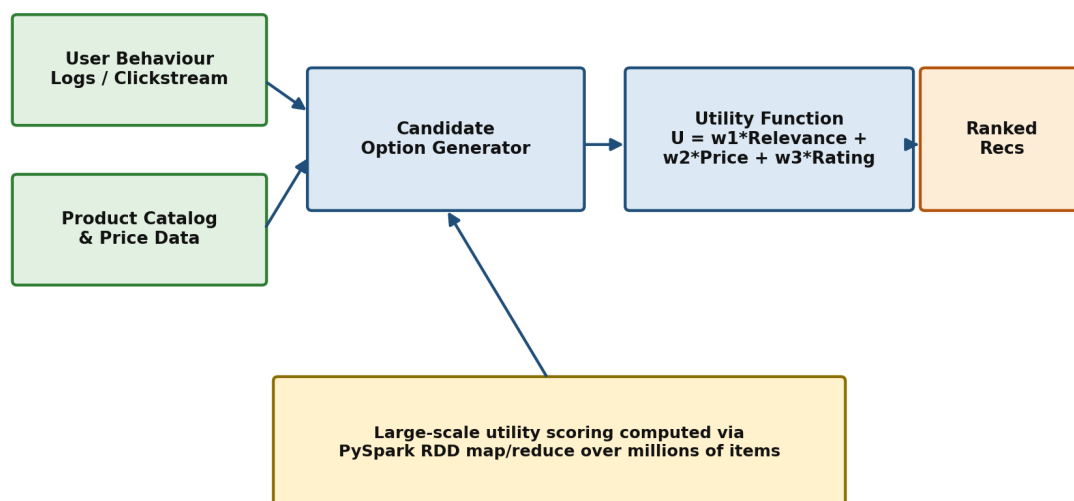


Fig. 3 — Utility-based agent pipeline for personalized product ranking.

3.5 Conclusion

The utility-based design allows the agent to make graded, preference-sensitive decisions rather than binary goal checks, and PySpark's distributed RDD operations make it possible to compute these utility scores across catalog- and user-scale data efficiently.

4. Learning Agent for Personalized Education System

4.1 Problem Identification

Students differ in pace, prior knowledge and preferred learning style. A static curriculum cannot adapt content difficulty to an individual learner, and hand-crafted rules cannot capture the many subtle patterns in how students respond to material. The problem requires a **learning agent** that improves its own performance-delivery policy over time, using feedback from the learner rather than fixed rules.

4.2 Agent Architecture

A classical learning agent has four components, all present in this design:

- **Performance element:** selects and delivers content/questions to the student based on the current model of their ability.
- **Critic:** evaluates learner responses against the desired learning outcome (e.g. mastery of a topic) and produces a feedback signal.
- **Learning element:** updates the internal model (e.g. difficulty-adaptation parameters, knowledge-tracing model) using the critic's feedback.
- **Problem generator:** suggests new, slightly challenging exercises to explore areas the student hasn't yet mastered, balancing exploration and exploitation.

4.3 Role of PySpark, RDDs and Distributed Data

A large e-learning platform generates continuous interaction logs (attempts, time spent, correctness) from thousands of concurrent learners. These logs are ingested as PySpark RDDs and processed with map/groupByKey transformations to build per-student and per-topic performance summaries. Periodic distributed retraining jobs (e.g. using Spark MLlib) recompute the difficulty-adaptation and knowledge-tracing models across the entire learner population, so improvements learned from one group of students propagate to the whole platform. RDD partitioning by student cohort enables this retraining to run in parallel across the cluster instead of sequentially per student.

4.4 Tools Used

- PySpark RDDs / Spark MLlib for distributed model retraining
- Learning-analytics event pipeline (interaction logging)
- Adaptive content-sequencing engine (performance element)
- Dashboard for tracking learner mastery over time

Learning Agent Architecture — Personalized Education System

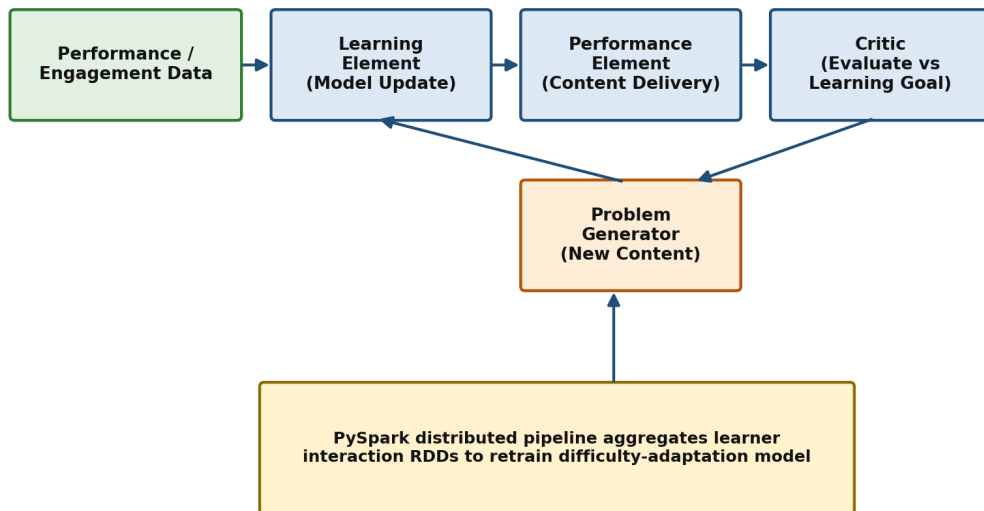


Fig. 4 — Learning agent components: performance element, critic, learning element, problem generator.

4.5 Conclusion

The learning-agent architecture allows the education platform to continuously refine how it teaches each student, and distributed PySpark processing ensures that this adaptation can be learned from — and delivered to — a very large learner population efficiently.

5. Multi-Agent System for Disaster Management

5.1 Problem Identification

Disaster response (floods, earthquakes, fires) involves many geographically distributed information sources and decision points — sensors detecting the event, teams needing to communicate, and responders needing to be dispatched — no single centralized agent can realistically sense and act everywhere at once. This problem is naturally suited to a **multi-agent system (MAS)**, where specialized agents each handle part of the task and coordinate toward a shared response goal.

5.2 Agent Architecture

- **Sensor agents:** monitor flood levels, seismic activity, and weather feeds at distributed field locations.
- **Communication agents:** relay percepts and alerts between field agents and the central coordination layer, handling network partitioning gracefully.
- **Decision agents:** fuse incoming reports to assess severity and allocate limited resources (rescue teams, medical supplies).
- **Response agents:** execute the dispatch of rescue teams and track task completion, reporting status back to decision agents.
- **Coordination mechanism:** a shared blackboard / negotiation protocol resolves conflicting resource requests between agents (e.g. contract-net protocol for task allocation).

5.3 Role of PySpark, RDDs and Real-Time Data Fusion

Disaster scenarios produce high-velocity, multi-source data (thousands of sensor readings per second across a region). PySpark Structured Streaming ingests this as continuous micro-batch RDDs, partitioned geographically so each partition can be processed on the nearest available compute node — reducing latency for time-critical alerts. `reduceByKey()` operations fuse readings from multiple sensor types (flood + seismic + weather) into a unified severity score per zone, which decision agents then use for resource allocation. This distributed fusion is essential because no single machine could process the volume and velocity of multi-region sensor data during an active disaster within the required response time.

5.4 Tools Used

- PySpark Structured Streaming / RDDs for distributed sensor-data fusion
- Multi-agent communication framework (message-passing / contract-net protocol)
- GIS mapping for geographically-aware task allocation
- Alerting and dispatch system integrated with emergency services

Multi-Agent System Architecture — Disaster Management

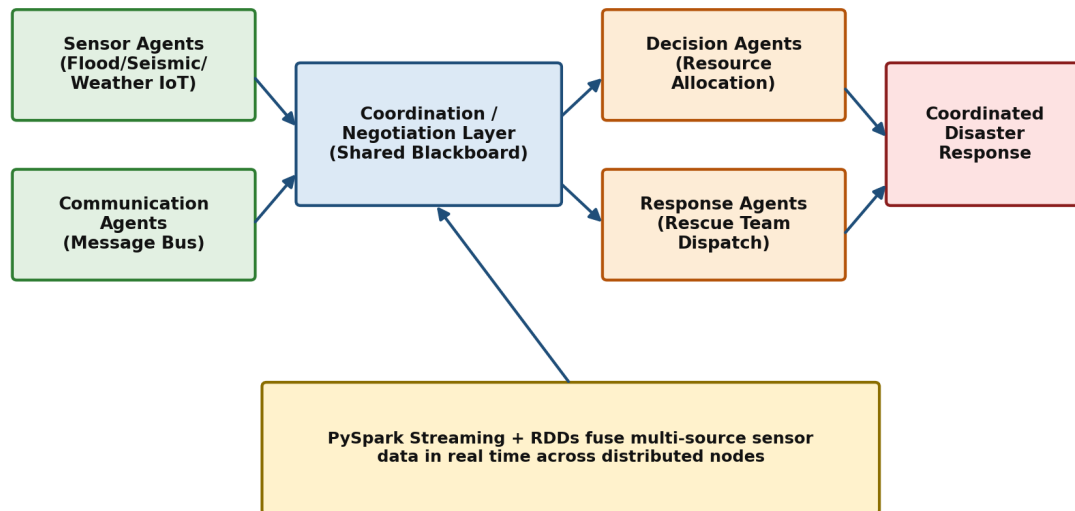


Fig. 5 — Multi-agent coordination for real-time disaster detection and response.

5.5 Conclusion

Decomposing disaster management into cooperating sensor, communication, decision and response agents — backed by PySpark's distributed stream processing — allows the system to fuse real-time data at scale and coordinate a faster, more resilient emergency response than any single centralized agent could achieve.

6. Rubric Alignment Summary

The table below maps each solution's content back to the grading rubric so the presentation delivery can be checked against all five weighted criteria before submission.

Criteria	Wt.	How each solution addresses it
Technical Content Accuracy	30	Each agent type (reactive, goal-based, utility-based, learning, multi-agent) is defined precisely
Problem Identification	25	Each section opens with an explicit problem statement showing why that agent type — and not
Use of Tools	20	Every topic explains concretely how PySpark RDD operations (map, filter, reduceByKey, join
Communication	15	Structured headings, bullet points, and diagrams are provided to support a clear, organized s
Professionalism	10	Consistent formatting and a conclusion for each topic model professional technical delivery; p

Presentation tip: when presenting, spend roughly equal time on each topic (assuming one topic per 10-mark question), lead with the problem statement, walk through the diagram, then close with the PySpark/RDD integration point — this mirrors the rubric's weighting and signals strong technical accuracy and problem identification to the evaluator.